

Trabalho 1 de AV3 - Arctic Network

Equipe D: Davi Dias, João Pedro Brasil, Lucas
Lustosa

Disciplina: Resolução prob com Grafos

Professor: Me Ricardo Carrubi

Modelagem do Problema como Grafo

Abstração e Conversão do Enunciado

- **Vértices (V):** Os P postos avançados mapeados pelas coordenadas cartesianas (x, y) fornecidas pela entrada.
- **Arestas (E):** Todos os caminhos de comunicação bidirecionais possíveis entre qualquer par de postos.
- **Grafo Completo (Clique):** Como qualquer posto pode potencialmente se comunicar com qualquer outro, modelamos a topologia como um grafo completo. O número total de arestas geradas é dado por:
 - $E = (P(P-1))/2$
- **Pesos das Arestas (W):** A distância física linear entre as coordenadas, calculada via Distância Euclidiana:
 - $W = \text{raiz}\{(x_1 - x_2)^2 + (y_1 - y_2)^2\}$

Estratégia Algorítmica — Kruskal Adaptado

Abstração e Conversão do Enunciado

- **Justificativa:** Diferente do algoritmo de Prim (que expande de forma radial a partir de uma origem fixa), a estratégia gulosa do Kruskal analisa as arestas globalmente, do menor para o maior peso, permitindo controlar diretamente o agrupamento de componentes isolados.
- **Estruturas de Suporte**
 - **Edge (Aresta):** Classe imutável encapsulando os vértices e peso, implementando a interface *Comparable* para ordenação natural nativa. Conta com os métodos de acesso abstrato *either()* e *other(v)*.
 - **PriorityQueue (Min-PQ):** Fila de prioridade mínima que recebe todas as arestas e nos devolve o elemento de menor peso em tempo $O(1)$ através da operação de extração *.poll()*.

O Papel do Union-Find e Variação da MST

Gerenciamento de Componentes e Interrupção Precoce

- **Classe *UF* (Union-Find):** Estrutura de dados acoplada ao Kruskal para otimização de consultas de conectividade.
 - ***find(i)***: Busca o representante do conjunto aplicando a técnica de **Path Compression** (redução da altura da árvore para tempo quase constante).
 - ***connected(v, w)***: Valida se dois vértices já estão na mesma subárvore, impedindo ciclos.
 - ***union(v, w)***: Unifica os conjuntos utilizando a heurística de **Union by Rank**.
- **Variação de MST:**
 - Uma MST tradicional roda até unificar todo o grafo em um único componente (count = 1).
 - Como dispomos de S canais de satélite (que cobrem distâncias infinitas gratuitamente), nós não precisamos fechar uma árvore geradora única por rádio.
 - **Regra de Parada:** Interrompemos o laço do Kruskal assim que o contador interno do Union-Find atinge o número exato de satélites: *if (uf.count() == S) break;*
 - A última aresta de rádio processada dita o nosso alcance mínimo D.

Análise de Complexidade

Eficiência de Tempo e Espaço

- **Complexidade de Tempo:**
 - Cálculo das distâncias e população da Min-PQ: $O(P^2 \log P)$ pelas inserções na fila.
 - **Processamento das arestas no Kruskal:** $O(E * \alpha(V))$, onde α é a função inversa de Ackermann (crescimento infinitesimal, tratada como constante $O(1)$ na prática).
 - **Complexidade Total:** Domínio assintótico de $O(P^2 \log P)$.
- **Complexidade de Espaço:**
 - Estrutura dominante na memória: A Fila de Prioridade, consumindo $O(P^2)$ de espaço para guardar todas as combinações de arestas mapeadas.
- **Desempenho Prático:**
 - No pior caso estipulado pelo problema ($P = 500$), geramos cerca de 124750 arestas. O volume de operações é processado em milissegundos, superando com folga o limite de 1.0 segundo exigido pelo professor e pela Kattis.

Casos Especiais e Arq. do Repositório

Blindagem do Código e Conclusão

- **Caso Limite Solucionado ($S \geq P$):** Se houver igual ou mais satélites do que postos avançados, o sistema de rádio é descartado. O algoritmo intercepta essa condição na entrada e retorna $D = 0.00$ instantaneamente.
- **Tratamento de Entradas Sujas:** Implementação de `.trim().isEmpty()` para filtrar quebras de linha residuais do arquivo e evitar erros de parsing.
- **Leitura Híbrida Inteligente:** Atendimento estrito à organização de pastas exigida pelo professor. O código verifica dinamicamente os caminhos locais para abrir o arquivo `dados/entradas_do_problema.txt`. Se não encontrar o arquivo (ambiente do servidor online), ele chaveia automaticamente para a entrada padrão do sistema (`System.in`).
- **Resultado:** Solução robusta e com veredito de **ACCEPTED** na plataforma Kattis.

Submission 19741681

[Edit and resubmit](#)

DATE	PROBLEM	JUDGEMENT	RUNTIME	LANGUAGE	TEST CASES
07:54:20 ✓ ✓	Arctic Network	✓ Accepted	0.18 s	Java	2/2

[Submitted Files](#) [Submission history](#)

Files submitted

Mainclass: `Main`

Main.java

[Download Main.java](#)

```
1 // AV3 - TRABALHO 1 - EQUIPE D - GRAFOS - PROFESSOR RICARDO CARRUBI
2 import java.io.BufferedReader;
3 import java.io.File;
4 import java.io.FileReader;
5 import java.io.IOException;
6 import java.io.InputStreamReader;
7 import java.util.PriorityQueue;
8 import java.util.Locale;
9 import java.util.StringTokenizer;
10
11 public class Main {
12
13     static class Edge implements Comparable<Edge> {
14         private final int v, w;
15         private final double weight;
16
17         public Edge(int v, int w, double weight) {
18             this.v = v;
19             this.w = w;
20             this.weight = weight;
21         }
22
23         public int either() {
24             return v;
25         }
26
27         public int other(int vertex) {
28             if (vertex == v) return w;
29             else return v;
30         }
31     }
32 }
```